Pietro Pizzoccheri 10797420 [team leader]

Lorenzo Bardelli 10831941

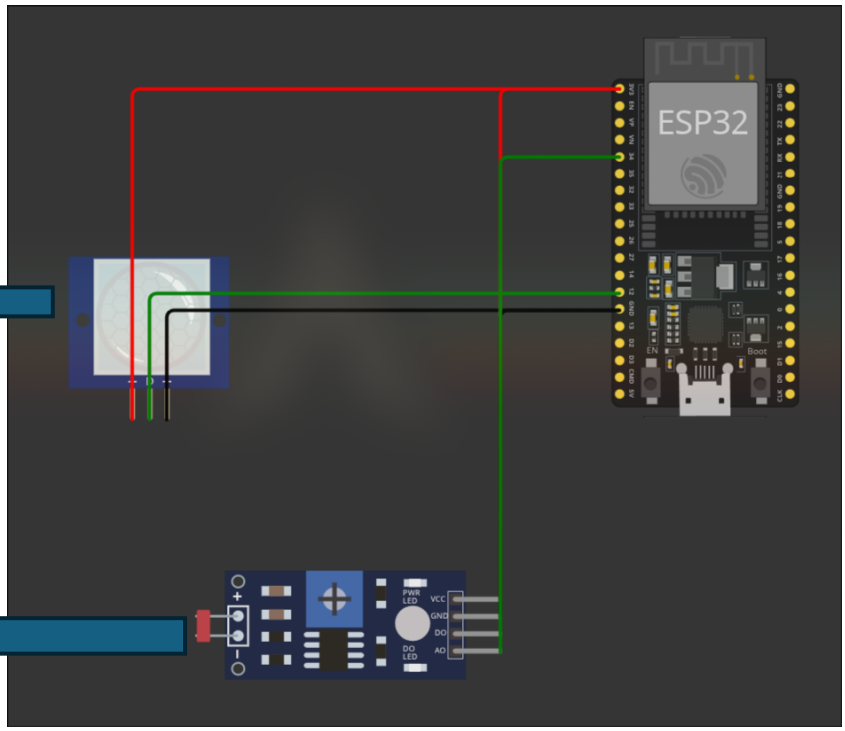
Internet of Things – First Challenge

Project Topics

The project consists of developing a simple motion sensor and lighting node using an ESP32 board paired with the PIR motion sensor and LDR photoresistor, using the ESP-NOW protocol for communicating between nodes and the central sink.

Schematics

Name	Description
GND	Ground
OUT	Output (digital)
VCC	Supply voltage



Name	Description
VCC	Positive power supply
GND	Ground
DO	Digital output
AO	Analog output

Code review

Constants, global variables and functions prototypes

```
#include <WiFi.h>
#include <esp_now.h>

/*
  Internet of Things - First Challenge
  Project Title: Smart Home Motion & Light Sensor
  Pietro Pizzoccheri: 10797420
  Lorenzo Bardelli: 10831941
*/

#define TEAM_LEADER_CODE 10797420
#define SLEEP_TIME_SEC ((20 % 50 + 5) / 10.0)

#define S_uS_CONV 1000000 // conversion factor from us to s

// sensor Pinout
#define PIR_PIN 12
#define LDR_PIN 34

long long int startTime;

// broadcast address required by the challenge: FF:FF:FF:FF:FF:FF
uint8_t broadcastAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};

esp_now_peer_info_t ESP_sink; // peer data

//  functions' prototypes  //
void wifiInit();
void onDataSent(const uint8_t *mac_addr, esp_now_send_status_t status);
void sendMessage(String message);
```

In this first part of the code, all constants and global variables are defined. To maintain code cleanliness, function prototypes are defined before the actual function implementation.

Functions

```
void setup()
{
    startTime = micros();
    // open serial port: 115200
    Serial.begin(115200);
    Serial.printf("[0] -- Booting\r\n");

    // sensor configuration
    pinMode(PIR_PIN, INPUT);
    pinMode(LDR_PIN, INPUT);

    Serial.printf("[%d] -- Setup completed\r\n", (int)(micros() - startTime));
}
```

The setup function contains all the instructions that must be executed once at every boot of the board. Every time the board starts, the startTime variable is initialized with the current board uptime in microseconds. This value is worth noting as it is not reset after the board enters deep sleep. Additionally, serial communication and board pins are initialized during this step.

```
void loop()
{
    //  loop will be executed only once per cycle due to deep sleep call  //

    // fetch data from sensors
    bool motionDetected = digitalRead(PIR_PIN) == HIGH;
    int luminosity = analogRead(LDR_PIN); // Reads 0-4095 on ESP32

    // format the message according to requirements
    String motionStr = motionDetected ? "MOTION_DETECTED" : "MOTION_NOT_DETECTED";
    String message = motionStr + "-LUMINOSITY:" + String(luminosity);

    Serial.printf("[%d] -- Sensor reading complete: %s\r\n", (int)(micros() - startTime), message.c_str());

    // configure WiFi for transmission and send message to sink
    wifiInit();
    Serial.printf("[%d] -- Wifi initialized\r\n", (int)(micros() - startTime));

    sendMessage(message);
    Serial.printf("[%d] -- Message sent\r\n", (int)(micros() - startTime));

    delay(50); // small delay to ensure transmission completes

    WiFi.mode(WIFI_OFF);
    Serial.printf("[%d] -- WiFi turned off\r\n", (int)(micros() - startTime));

    Serial.printf("[%d] -- Entering deep sleep for %.1f seconds...\r\n", (int)(micros() - startTime), SLEEP_TIME_SEC);
    Serial.flush(); // flush serial buffer before entering deep sleep

    // set wakeup timer and sleep
    esp_sleep_enable_timer_wakeup(SLEEP_TIME_SEC * S_uS_CONV);
    esp_deep_sleep_start();
}
```

The loop function is usually executed multiple times, but due to the board entering deep sleep at the end of each cycle, it's only executed once per power cycle. During the loop(), the board reads from the sensors using digitalRead on the corresponding input pins, initialises the WiFi module, sends the message containing the readings to the sink node, and finally turns off the WiFi module before entering deep sleep.

```
void wifiInit()
{
    WiFi.mode(WIFI_STA);
    WiFi.setTxPower(WIFI_POWER_2dBm);
    esp_now_init(); // init ESP-NOW communication protocol

    esp_now_register_send_cb(onDataSent); // set sending callback

    // peer registration
    memcpy(ESP_sink.peer_addr, broadcastAddress, 6);
    ESP_sink.channel = 0;
    ESP_sink.encrypt = false;

    esp_now_add_peer(&ESP_sink); // add sink to peers
}
```

This snippet demonstrates how the WiFi transmitter is turned on and the communication protocol is set up.

Initially, the WiFi module is switched on and placed in Station mode. Transmission power is then set to 2dBm power for simulation purposes, assuming the room is small enough to have the sink node nearby.

Callback functions are then set for sending messages. Finally, the sink is registered as a peer in the ESP-NOW communication protocol, using the broadcast MAC address, channel 0, and an unencrypted connection.

Callback functions

```
void onDataSent(const uint8_t *mac_addr, esp_now_send_status_t status)
{
    (void)mac_addr;
    Serial.printf("Message status: ");
    Serial.printf(status == ESP_NOW_SEND_SUCCESS ? "SENT\r\n" : "ERROR\r\n");
}

// send message through WiFi with ESP-NOW protocol
void sendMessage(String message)
{
    uint8_t *addr = broadcastAddress;
    uint8_t *data = (uint8_t *)message.c_str(); // casting string to uint8_t

    int len = message.length() + 1;
    esp_now_send(addr, data, len);
}
```

These functions manage the handling and dispatch of messages.

When a message is sent via esp_now_send() within sendMessage(), the onDataSent() function is invoked. This function appends the message string to the data and transmits it to the broadcast address.

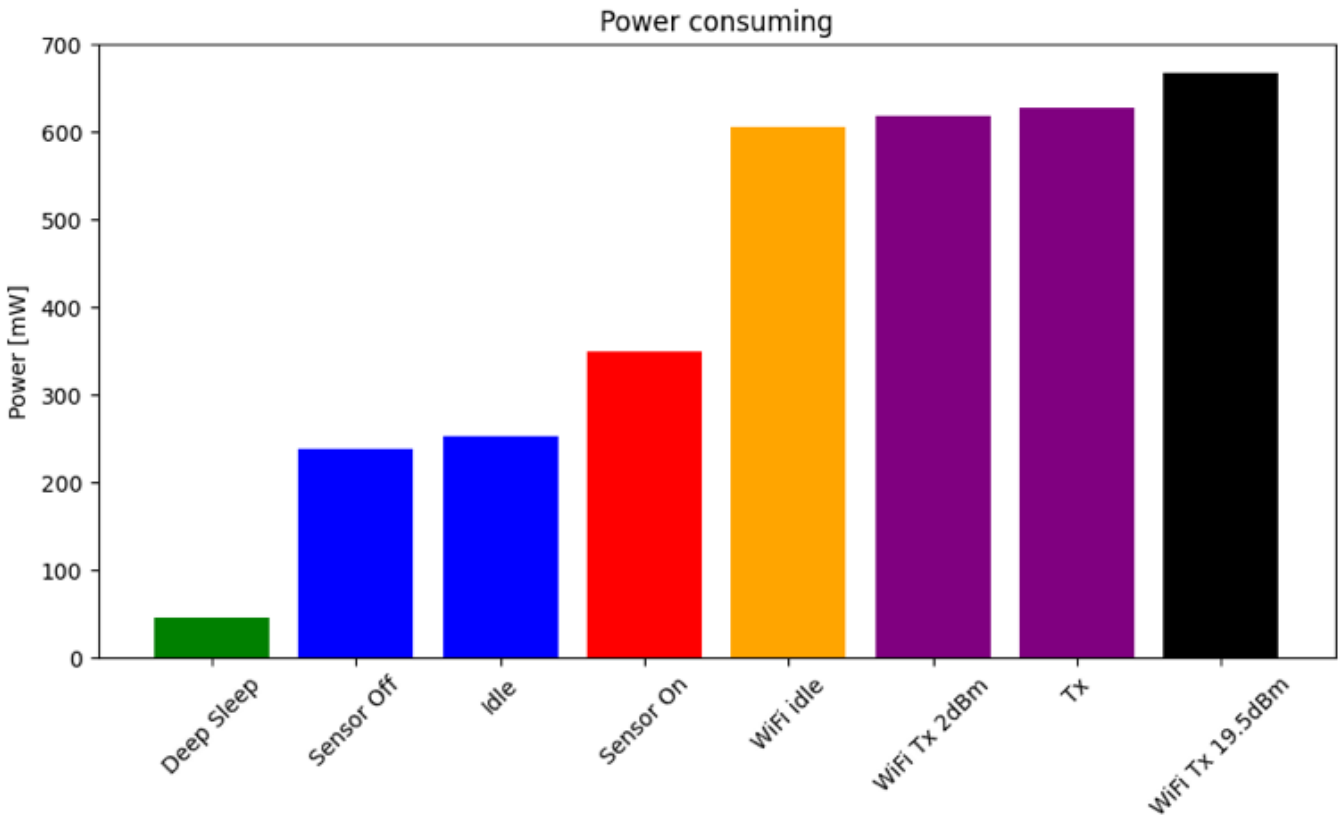
Since the node's objective is to notify the sink, there is no need to handle data reception. The sink will appropriately respond by turning the light on or off as needed.

Energy Consumption and Average Power Estimation

The results below are from our notebook, used to estimate energy and power consumption. To calculate power consumption, we filtered the data points based on empirical observations of the plotted data. In our case, we don't include the Transmission power at 19.5 dBm because we only use the component at 2 dBm.

Average Power Consumption

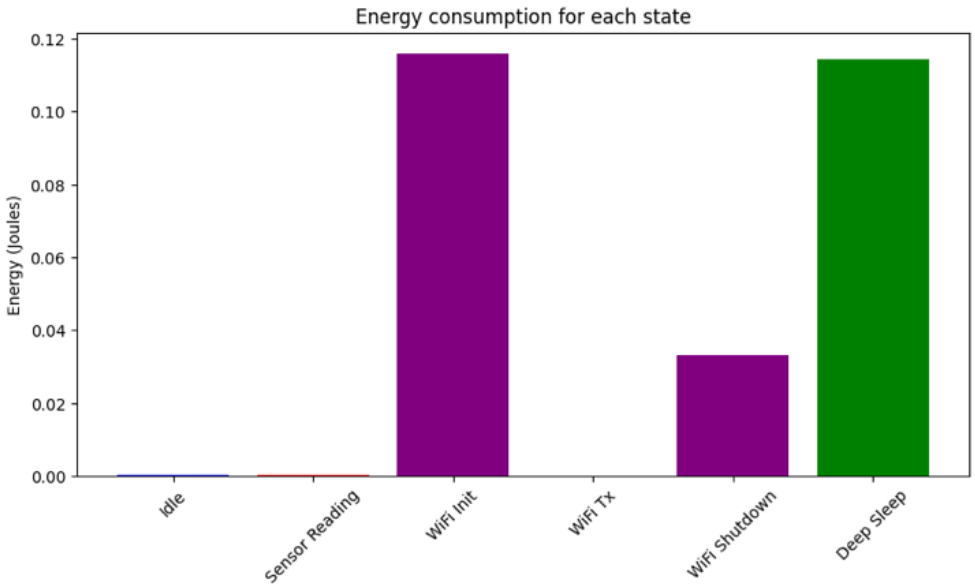
Average Power Consumption for each state:



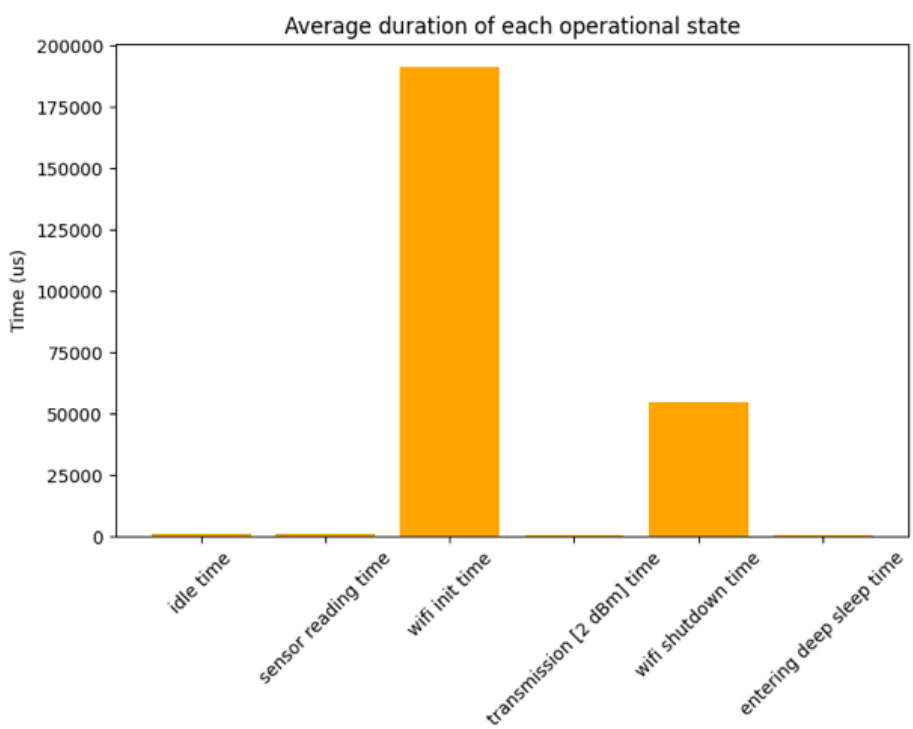
Deep Sleep	45.777 mW
Idle	252.222 mW
Tx	627.259 mW
Sensor On	350.162 mW
Sensor Off (idling)	238.019 mW
WiFi idle	605.889 mW
WiFi Tx 2dBm	617.883 mW
WiFi Tx 19.5dBm	667.411 mW

Average Energy Consumption

Data related to the time spent in each operational state was obtained by using the micros() function. This function returns the number of microseconds elapsed since the system was powered on. However, micros() does not reset when the system enters deep sleep, so for deep sleep, we used a hardcoded value of 2.5 seconds ((20%50) + 5) / 10.



Idle	0.240 mJ
Sensor Reading	0.286 mJ
WiFi Init	115.894 mJ
WiFi Tx	0.101 mJ
WiFi Shutdown	33.096 mJ
Deep Sleep	114.443 mJ



Battery Life Estimation

$$duty\ cycles = \frac{J_{battery}[J]}{J_{cycle}[J]} = 65970$$

$$battery\ life = duty\ cycles * \frac{time_{simulation}\ [s]}{3600\left[\frac{s}{h}\right]} = 50{,}35h$$

Possible Improvements

In the base implementation, the sensor wakes up every 2.5 seconds, reads the sensors, and every time it initializes the Wi-Fi module to transmit a message even if no motion is detected.

According to our energy estimation in the graph above, Wi-Fi is the most consuming per unit of time, making up most of the energy consumption during the cycle (also, deep sleep consumes a lot, but it is much longer in terms of time).

As a solution to drastically reduce energy consumption without compromising the main goal of the system (turning on lights when a person is present, considering also luminosity value), we propose to modify the code to only activate the Wi-Fi when motion is detected and ambient light is below a certain threshold (we used 2000 as a value, but that's just for reference; in a real implementation, the value must be fine-tuned).

In code terms, we simply check these values, and if they are not matched, we skip the wifiInit() and message sending logic.

```
Serial.printf("[%d] -- Sensor reading complete: %s\r\n", (int)(micros() - startTime), message.c_str());

if (motionDetected && luminosity < 2000) // only send message if motion is detected and luminosity is low, to save energy
{
    // configure WiFi for transmission and send message to sink
    wifiInit();
    Serial.printf("[%d] -- Wifi initialized\r\n", (int)(micros() - startTime));

    sendMessage(message);
    Serial.printf("[%d] -- Message sent\r\n", (int)(micros() - startTime));

    delay(50); // small delay to ensure transmission completes

    WiFi.mode(WIFI_OFF);
    Serial.printf("[%d] -- WiFi turned off\r\n", (int)(micros() - startTime));
}

Serial.printf("[%d] -- Entering deep sleep for %.1f seconds...\r\n", (int)(micros() - startTime), SLEEP_TIME_SEC);
```

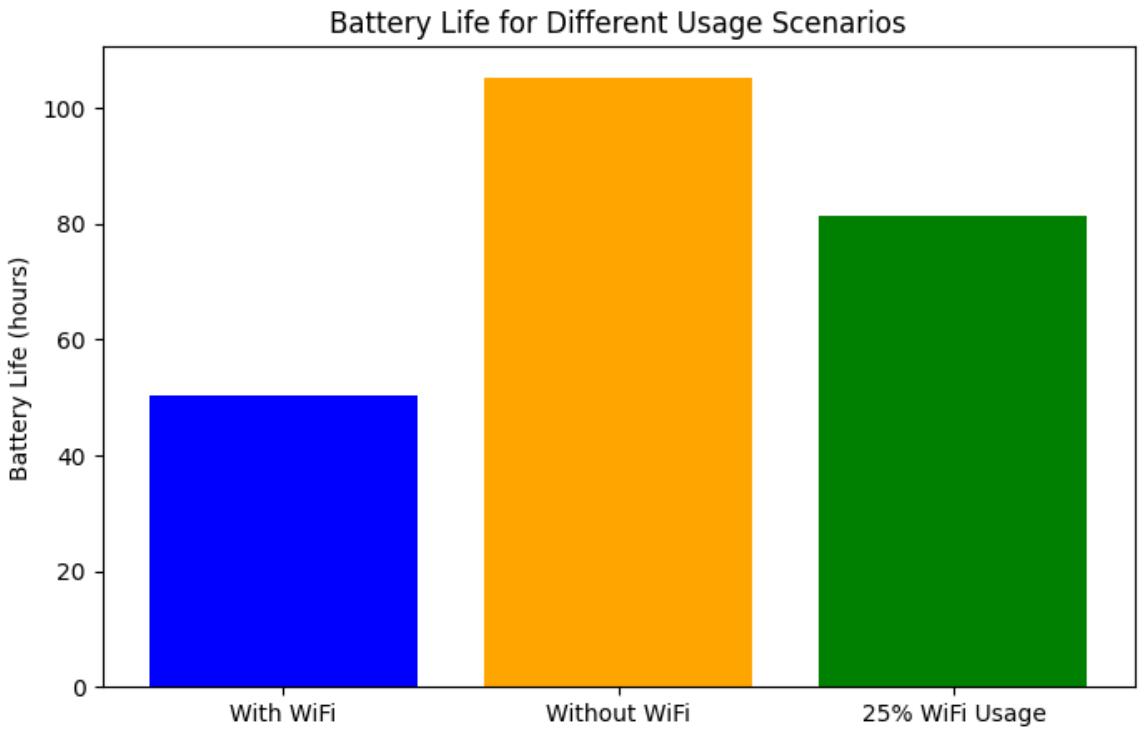
By skipping this phase, the energy consumed when the space to check is empty is reduced. To quantify this improvement, we simulated a scenario where the luminosity and motion are detected for 25% of the time (maybe an overestimation).

By doing this, we increased battery life by 61.80%, equal to 31.12 hours of lifetime.

For better insights on this part, look at the final part of this notebook:
https://github.com/pitesse/IoT/blob/master/challenge1/energy_estimation/energy_estimation.ipynb

Other improvements may be to use a longer sleep cycle (up to 4/5 seconds), even if this may impact UX since it would be better that the light turns on as soon as the checks are matched. So, this is a trade-off.

Also, we may consider less consuming technologies since for this use case, it isn't acceptable to have 100/150 hrs of battery life.



Hours gained	31.12 h
Lifetime increase	61.80%